

Fixed-Unit-Domain Convention for Triply-Periodic Minimal Surface Visualisation

engvis

June 19, 2026

Abstract

When the user varies the period parameter k of a triply-periodic minimal surface (TPMS) inside a fixed $[-1, 1]^3$ unit cube, the visual density of the isosurface changes predictably: larger k means more periods visible. This is the expected behaviour—the unit cube serves as a fixed “window” through which the periodic pattern is observed.

This note documents the convention and explains why an adaptive domain (scaling the cube to keep one period always visible) was explicitly rejected: it removes the user’s visual feedback for the period slider, making it impossible to judge whether the pattern is getting denser or sparser.

1 Design Principle

The engvis viewer maintains a *fixed unit domain* for all implicit surfaces (sphere, torus, TPMS, custom Rhai expressions). The sampling grid for Marching Cubes 33 is always

$$\Omega = [-1, 1]^3, \tag{1}$$

and the bounding wireframe cube is likewise fixed at ± 1 . The orbit camera is initialised to fit this same box, so the user always views the same physical volume.

When the user drags the **Period k** slider from $k = 4$ to $k = 8$, the trigonometric argument changes from $4x$ to $8x$, putting twice as many periods inside Ω (Figure 1). The surface visibly “densifies”—which is exactly the feedback the user expects from a parameter control.

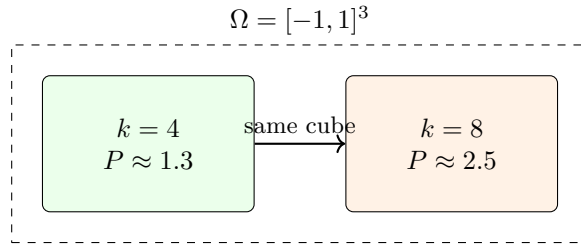


Figure 1: Left: $k = 4$ —about 1.3 periods inside the unit cube. Right: $k = 8$ —about 2.5 periods. The cube stays the same; the surface visibly gets denser.

2 Why Not an Adaptive Domain?

An alternative considered and rejected was to scale the domain width $D(k) = 2\pi/k$ so that exactly one period is always visible, then scale the wireframe and camera to match (the “auto-scale” approach). This keeps the visual proportion constant but *destroys the user experience* of the period slider:

- The surface looks **exactly the same** for all k values—the slider has no visible effect.
- MC 33 accuracy degrades when $D(k) < 1$ because the same R^3 grid cells are stretched over a smaller volume, increasing the effective resolution unnecessarily.
- The wireframe cube shrinks or grows simultaneously with the slider, defeating the purpose of having a fixed reference frame.

The chosen convention (fixed unit domain) is also simpler: no extra parameters, no camera/logic coupling between domain sizing and clipping.

3 Edge Cases

- k **very large** ($\rightarrow 10$). $P \approx 3.2$ periods fit inside Ω . The surface becomes very dense; the mesh may have many small triangles but remains inside the wireframe. `clip_to_unit_ball` provides manual control over which portion of the dense surface is visible.
- k **very small** ($\rightarrow 0.5$). $P \approx 0.16$. Only a fraction of a period is visible—the surface looks like a smooth patch. This is acceptable because the slider covers the full range deliberately.

4 Boundary Vertex Clamping

When MC 33 evaluates f at domain boundaries $(\pm 1, y, z)$ and the neighbouring interior sample has opposite sign, the face carries a zero-crossing. The linear interpolation factor $t = \frac{v_a}{v_a - v_b}$ can land fractionally outside $[-1, 1]$ when v_a (the boundary sample) is polluted by floating-point noise. Without correction, a few vertices end up at $a \in \{x, y, z\} = 1.0000001$, visually “overflowing” the wireframe cube.

A simple post-extraction clamp fixes this in $O(N)$ with zero perceptual impact:

```
let (mut pos, idx) = marching_cubes::extract(f, ...);
for p in &mut pos {
    for v in &mut *p {
        if *v < -1.0 { *v = -1.0; }
        if *v > 1.0 { *v = 1.0; }
    }
}
```

Only coordinates that deviate by a few machine epsilons are touched; the topology (*which* edge carries an intersection) is unaffected.

5 Adaptive Resolution

A deeper problem surfaces when the implicit function contains features thinner than ≈ 3 grid cells. The MC 33 grid cannot resolve sub-cell topology, producing degenerate triangles and visually broken meshes—most notably when the torus minor radius is dragged below ≈ 0.05 at the default resolution $R = 64$.

The fix is a heuristic **auto-resolution bump** that runs before every mesh build. For each surface type a minimum resolved feature size is estimated:

The requested resolution is clamped to `[user  $R$ , 512]`. When the bump activates the console prints a diagnostic line:

```
[MC33] auto-bump res 64 -> 160 (feature 0.040)
```

The user-visible slider still shows 64; the bump is transparent.

Table 1: Minimum feature-size estimates used for resolution bumping.

Surface	Feature	Needed R
Torus	$2r$ (tube diameter)	$R \geq 6/(2r)$
TPMS	π/k (half-period)	$R \geq 6k/\pi$
Sphere & others	0.5 (generous)	always $\geq R_{\text{user}}$

6 Code Layout

The two fixes sit in neighbouring lines of `src/main.rs`:

```
// 1. Boundary clamp (in build_mc33_mesh)
for p in &mut pos {
    for v in &mut *p {
        if *v < -1.0 { *v = -1.0; }
        if *v > 1.0 { *v = 1.0; }
    }
}

// 2. Adaptive resolution (in build_scene)
let effective_res = {
    let min_feature = match &self.source { ... };
    ((6.0 / min_feature) as usize).max(self.mc_res).min(512)
};
```

Wireframe & camera are static—same fixed-unit-domain convention as before—so the reference frame the user sees remains unchanged regardless of internal resolution adjustments.